EXPERIMENT – 6

Title: Prototype Multithreaded Server

Objective:

To implement a multithreaded server that can handle multiple client connections at the same time.

Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

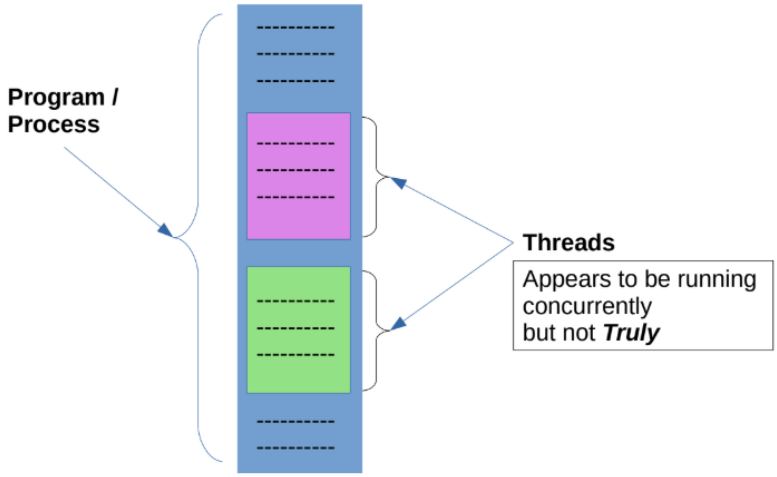
A thread is a small unit of execution inside a program that can run independently. A single process can have multiple threads, and each thread performs its own task.

In networking, using threads helps a server to handle multiple clients at the same time. Instead of serving one client and then moving to another, the server creates a separate thread for each client connection.

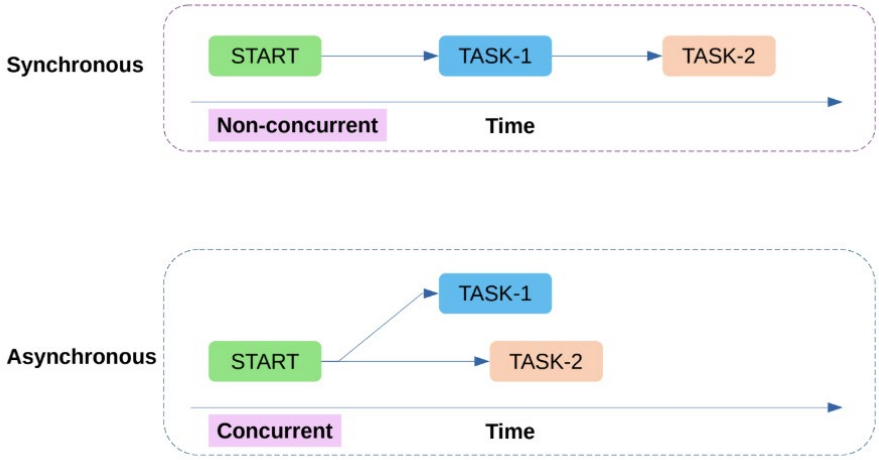
There are two types of execution: synchronous and asynchronous. In synchronous execution, tasks are performed one after another, while in asynchronous execution, multiple tasks appear to run at the same time.

In this experiment, a multithreaded TCP server is created where each client is handled by a separate thread. The server sends data to multiple clients and receives responses from them independently.

Concept of Thread



Synchronous & Asynchronous Thread



Procedure / Algorithm

Server Side

1. Import required modules.
2. Create TCP server socket and start server.
3. Wait for client connection and accept it.
4. Create a new thread for each client.
5. Send data to client and receive response.
6. If client sends "close", terminate that thread

Client Side

1. Import required modules.
2. Create TCP client socket.
3. Connect to server.
4. Receive and display message.
5. Send acknowledgment to server.
6. If user exits, send "close" and terminate connection.

Program Code

Server Program:

```python
import netpack as npk
import random
import threading
import time

PORT=9999
server_ip="127.0.0.1"
ADDR=(server_ip, PORT)
client_details={}
client_id=1

def clientHandle(client_socket, addr, id):
    lock = threading.Lock()
    global client_details

    while True:
        try:
            lock.acquire()
            val=random.randint(0,100)
            npk.send_data_to_tcp_client(client_socket,val)
            print("[SEND] to client {} Data {}".format(addr, val))

            data=npk.read_data_from_tcp_client(client_socket)
            if data=="close":
                npk.close_socket_connection(client_socket)
                print("[=] TCP Client - {} requesting to close connection".format(addr))
                break
            else:
                print("[RECEV] {} from Client - {}".format(data, addr))
        except (ConnectionResetError,BrokenPipeError):
            print("[ERROR] Client - {} connection".format(addr))
            break
        finally:
            lock.release()

    client_details[id].append(time.ctime(time.time()))
    print("Closed Thread for client - {} & port - {}".format(addr[0], addr[1]))

# Step-1
server_socket=npk.create_tcp_socket()

# Step-2
npk.start_tcp_server(server_socket, ADDR)

# Step-3
print("[WAIT] Server waits for client")

while True:
    try:
        (client_socket, addr)=npk.accept_tcp_client(server_socket)
        print('[CONNECTED] Client with IP : {} & PORT : {}'.format(addr[0], addr[1]))
        client_thread = threading.Thread(target=clientHandle,
                                         args=(client_socket, addr, client_id),
                                         daemon=True)
        client_thread.start()

        client_details[client_id]=[addr, time.ctime(time.time())]
        client_id +=1

        print("*"*100)
        for id, client_info in client_details.items():
            print(f"Client - {id} connection details - {client_info}")
        print("*"*100)

    except KeyboardInterrupt:
        break

print("*"*100)
for id, client_info in client_details.items():
    print(f"Client - {id} connection details - {client_info}")
print("*"*100)
npk.close_socket_connection(server_socket)
```

Client Program:

```python
import netpack as npk

PORT=9999
client_ip="127.0.0.1"

server_ip=client_ip
ADDR=(server_ip, PORT)


# Step-1
client_socket=npk.create_tcp_socket()

# Step-2
npk.connect_to_tcp_server(client_socket, ADDR)
print('[CONNECTED] Server with IP : {} & PORT : {}'.format(ADDR[0],

while True:
    try:
        data=npk.read_data_from_tcp_server(client_socket)

        print('[=] Message from server : {}'.format(data))

        flag=input('Close connection y/n : ')
        if flag=='y' or flag=='Y':
            npk.send_data_to_tcp_server(client_socket,"close")
            npk.close_socket_connection(client_socket)
            print("[=] Requested for closing connection")
            break
        else:
            npk.send_data_to_tcp_server(client_socket,"+ACK")
    except KeyboardInterrupt:
        npk.send_data_to_tcp_server(client_socket,"close")
        break
    except (ConnectionResetError,BrokenPipeError): # If server conn
        npk.send_data_to_tcp_server(client_socket,"close")
    except:
        break

npk.close_socket_connection(client_socket)
```

Output:

Server Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-6$ python3 6_Multithreaded_TCP_server.py
[WAIT] Server waits for client
[CONNECTED] Client with IP : 127.0.0.1 & PORT : 44610
*****************************************************************************************
[SEND] to client ('127.0.0.1', 44610) Data 99
Client - 1 connection details - [('127.0.0.1', 44610), 'Tue Mar 24 18:17:17 2026']
*****************************************************************************************
[RECEV] +ACK from Client - ('127.0.0.1', 44610)
[SEND] to client ('127.0.0.1', 44610) Data 20
[RECEV] +ACK from Client - ('127.0.0.1', 44610)
[SEND] to client ('127.0.0.1', 44610) Data 2
[RECEV] +ACK from Client - ('127.0.0.1', 44610)
[SEND] to client ('127.0.0.1', 44610) Data 43
```

Client Side:

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-6$ python3 6_TCP_client.py
[CONNECTED] Server with IP : 127.0.0.1 & PORT : 9999
[=] Message from server : 99
Close connection y/n : n
[=] Message from server : 20
Close connection y/n : n
[=] Message from server : 2
Close connection y/n : n
[=] Message from server : 43
Close connection y/n :
```

Result

The multithreaded server program was executed successfully and multiple clients were handled simultaneously.

Conclusion

The multithreaded server program was executed successfully and multiple clients were handled simultaneously.